245

Basic Idea of Corpus-Supported Approach to Question Answering

Marcin SKOWRON Kenji ARAKI
Graduate School of Engineering, Hokkaido University

Abstract

We present a corpus-supported approach to question answering, which enables our system to classify a question category, formulate a query and verify answer candidates. The system uses the Internet to find information required to provide an answer. The role of the corpus is to support the system with knowledge on how to do it effectively using previously answered questions as the system "experience base". We conclude this paper by describing application of presented methods to our system.

1. Introduction

In recent years the amount of text available online has been rapidly increasing; in 2003, there were 3 billion web pages and their numbers continue to grow. While the amount of coverage and information amount increases, it has also become more difficult for the average Internet user to access high quality, specific information. We believe that the most natural approach for such a task would be to ask a question in natural language form and receive an exact answer, rather than a list of documents containing an inputted keyword. In our opinion an answer to a question answering system should resemble those given by humans. The current question answering systems fail to meet this requirement. Furthermore, they frequently use several linguistic resources like parsers, name entity recognizers, dictionaries etc., and as such, are difficult to transfer to other languages and depend heavily on the performance of such tools.

Our system differs from most question answering systems in its extensive usage of the corpus, consisting of questions with correct answers and question category classification, used to process a new question by acquiring several important features that support question classification, query formulation and answer candidates verification. Our system uses the Internet to find information required to provide an answer. The role of the corpus is to support the system with knowledge on how to do it effectively using previously answered questions as the system "experience base". We present a Query Pattern Generation Method, which for a given category and question syntax, forms a query that returns answers in a relatively undistorted set of passages. For question classification we use Support Vector Machine (SVM) [1], which is known to work well for sparse, high dimensional problems. Answer candidate verification is performed by applying the set of reduction rules that are automatically generated using the surface, syntactic and semantic features extracted from the answers from the corpus for a given question category. The length of the answer returned by our system is question-dependent; aimed to include only those words that are required.

2. Corpus Based Methods for Question Answering

2.1 Question Classification

In order to provide an answer to a user question from a large collection of texts, the system is required to impose some constraints on the scope of possible answers, an example of which is question classification. Our system performs question classification using the models generated by Support Vector Machine [2] trained on the corpus consisting of 5500 questions labeled manually by UIUC [3], with 6 coarse-grained and 50 fine-grained categories (Table 1). For our preliminary test we constructed vectors using text surface features (a bag-of-words) and trained our multi-class SVM using the one-versus-all approach.

Table 1: The coarse and fine-grained question categories

Coarse	Fine
ABBR	abbreviation, expansion
DESC	definition, description, manner, reason
ENTY	animal, body, color, creation, currency, disease, event, food, instrument, language, letter, other, plant, product, religion, sport, substance, symbol, technique, term, vehicle, word
HUM	description, group, individual, title
LOC	city, country, mountain, other, state
NUM	code, count, date, distance, money, order, other, percent, period, speed, temperature, size, weight

2.2 Query Pattern Generation

To reliably obtain an answer, the system needs to construct a query that returns it in a relatively undistorted set of passages. A query that is too precise does not generate a sufficient number of answer candidates to allow a frequency-based selection of an answer or does not return any documents at all. On the other hand, too general a query is likely to produce a distorted set of passages where an answer is relatively rarely provided. In our system a query is generated by joining a Question Text Pattern (QTP), a combination of words from a question, with a Surface Text Pattern (STP) [4], a word or non-letter character that connects an answer with a QTP. The aim of such a query is to receive a reply in as undistorted set of passages as possible.

To avoid the necessity to build and verify large numbers of permutations of words in a question we use Question Syntactic Representation (QSR), which reduces computational and extensive Internet usage requirements, while assuring that several potentially important question-elements combinations are being verified. The QSR is created using POS tags [5], which are later processed using the connection rules that allow linking specific combinations of tags like for example: Det+Adverb+Noun (/DT /JJ /NN = /NP).

Using QSR, the script produces several combinations of its elements. These combinations are sent one by one to the search engine along with the corresponding answer. The set of summaries from the first 200 web pages is collected, and the co-occurrence (positive hits) or lack (negative hits) of the particular query with an answer within one sentence window is counted. The QTP with the highest rate of

positive hits to negative ones are selected and stored. In the same process, using all sentences classified as a positive hit, the script extracts STP. The most frequent STP is joined with QTP and together form a Query Pattern (QP). This QP is associated with a particular question category and QSR, and then stored in the QP Repository.

For example, for the question, "Who was the first man on the Moon?" the script verifies several QSR element combinations and "/NP /IN /NP2 /VBD +" is found to form a reliable QTP like in the sentence, "[...]the first man on the Moon was Neil Armstrong[...]." (the plus sign in the QTP shows the expected place of answer candidate occurrence). In this case there are no additional characters between the QTP and an answer, so STP is not created. However, for another QTP "/NP /IN /NP2", the script finds in the sentence: "[...]Neil Armstrong, the first man on the Moon[...]." a STP which is ",". Thus, the new record in the QP Repository is formed in the following manner: QCat (HUM:individual), QSR (/WP /VBD /NP1 /IN /NP2 /?), QP (+, /NP /IN /NP2).

2.3 Answer Selection and Formulation

Based on information extracted from the corpus of previously answered questions, the system can be equipped with knowledge about the common elements, possible variation and other characteristic features of answers from a given question category. Our method applied to extract this knowledge includes examining the surface, syntactic and semantic features of answers for a particular question category. Surface features include: number of words, letter capitalization, non-letter characters, etc. A syntactic feature is a POS tag assigned to a word and a semantic feature is a hyponym category extracted for all nouns from the WordNet. Using these features, the script automatically generates a set of answer candidate verification rules, used to exclude candidates that are not related to a question category, or more specifically do not resemble any answer from the same category found in the corpus.

3. Corpus-Supported Question Answering System

The system accepts questions in natural language form (our first implementation is for English) and returns answers in question-dependent length. After the user question is inputted, the system consults the SVM models and assigns the most probable question category. Using question classification and QSR, the QP Repository is searched for a suitable QP, which is then used to form a query for a new question. This query is sent to the search engine. If the number of accessed passages does not reach 100, a query is gradually extended with synonyms of verbs, adjectives and adverbs (WordNet 2.0) until it reaches the set threshold. From accessed passages, the system extracts phrases that exist on the correct side of the QP in one sentence window. The answer candidate verification rules are applied to these phrases to exclude the ones that do not belong to a particular question category, after which a set of answer candidates is obtained. If the number of answer candidates does not reach 25, the system assumes that answer generation based on this set would not be reliable. To access a larger number of answer candidates, it uses the next best QP to form a new query, while gradually relaxing question category constraints. The result of the latter is the acceptance of answer candidates from other fine-categories, provided they stay in the same coarse category. With a sufficient number of answer candidates, the system counts the occurrence of particular ones and uses a voting method to determine those that consist of words that were also found in other candidates. The answer is then chosen and if possible, extended to guarantee that it has all the required words. Finally, if the user verifies an answer as a correct one, it forms a new entry in the corpus.

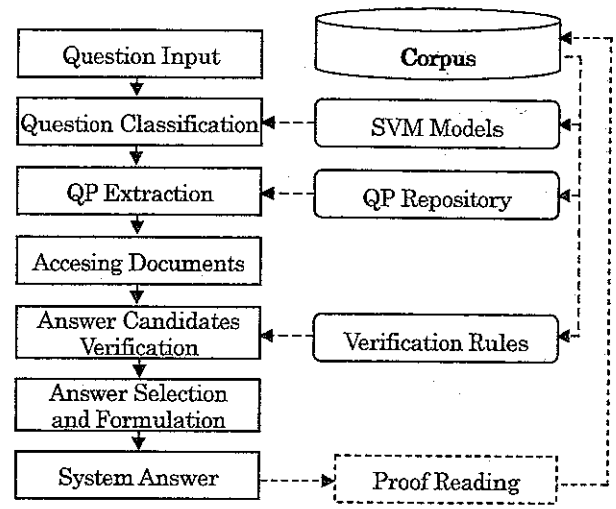


Fig. 1: The system flow-chart

4. Conclusion and Future Work

Efficient acquisition of knowledge from large collection of text documents like that of the Internet requires the appropriate tools. We believe that our approach may contribute to the improvement of question answering systems that would address the flaws inherent in search engines.

We proposed a corpus-supported question answering system that is able, by using information extracted from the corpus, to automatically classify a question category, generate a query and verify answer candidates. At present, the number and size of corpora that are applicable in these systems are limited. However, this should be overcome with the appearance of new systems that use the corpus-supported approach, and increase in number of evaluated materials (for example TREC QA tracks). Our future work includes extensive system testing and evaluation of the proposed methods to determine their individual contribution to the overall performance of the system.

References

(1) C. Cortes & V. Vapnik: "Support-Vector Network", Machine Learning 20, (1995).
(2) T. Joachims: "Making large-Scale SVM Learning Practical", MIT-Press, (1999).
(3) X. Li & D. Roth: "Learning Question Classifiers", In Proceedings of the 19th International Conference on Computational Linguistics (COLING'02), (2002).
(4) M. M. Soubbotin & S.M. Soubbotin: "Use of Patterns for detection of likely Answer Strings: A Systematic Approach Answer", Proceedings of the TREC-2002 Conference (2002).
(5) E. Brill: " Transformation-Based Error Driven Learning and Natural Language Processing: A Case Study in Part of Speech Tagging", Computational Linguistics (1995).